WPT Day 15 Evening

Abhishek Purohit · Wed 04 Nov 26 · PUT `/product/:id` with a parameterized `UPDATE`, edit-icon `useNavigate` that carries the product id, `useParams` + `GET /product/:id` to prefill the form, Axios `PUT`, Tailwind CSS as a utility layer on top of MUI, `StyledEngineProvider` CSS layers so Tailwind wins, and an active sidenav from `useLocation`

Table of Contents

1. PUT Update API
2. Edit Flow and useNavigate with ID
3. useParams and Prefill
4. Axios PUT and the Missing-ID Bug
5. Data Grids, AG Grid and What This CRUD Is Not
6. Tailwind CSS
7. Tailwind Install on Vite
8. CSS Layers
9. Dashboard, AppLayout and Theme
10. Active Sidenav
11. Products-List Header
12. Combined Worked Example
13. Quick Revision Checklist
14. Practice Exercises
15. Practice Answers

PUT Update API

Problem: the catalogue can create, read and delete — it cannot change a row

The morning session finished **C, R and D**. The backend already had:

Method	URL	Job
`POST`	`/product`	Insert a row
`GET`	`/product`	All rows
`GET`	`/product/:id`	One row
`DELETE`	`/product/:id`	Remove a row

What it did **not** have is **U** — update. The evening project is a copy of that stack (`crud-ui` + `crud-apis` , `node_modules` left behind, `npm install` in both). Frontend on **5173**, API on **7500**. The first job is a new Express handler that runs an `UPDATE` .

Mechanism: new values in the body, which-row in the URL

An update needs **two** pieces of data:

Piece	Where it travels	Example
New field values	`request.body`	`{ name, price, description, qty }`
Which product	`request.params.id`	`4` from `PUT /product/4`

The lecturer: *request body is still going to contain name, price, description and the quantity but this time your query is going to become the update query.* The `WHERE` clause is what makes it an update of **one** row, not a rewrite of the whole table.

`request.params` is the object of **every** URL parameter. You destructure the one you named in the route (`id`), the same way morning's `getById` / `deleteById` already do.

Parameterized `UPDATE` — do not paste values into the SQL string

Same SQL-injection rule as Day 8 insert. Do **not** write `SET name='${name}'`. Put `?` in the SQL and pass the values as a second argument.

```
const updateQry = `UPDATE product SET name=?, price=?, description=?, qty=? WHERE id=?
`;

const [result] = await sqlConnectionPool.query(updateQry, [
  name, price, description, qty, id,
]);
```

Five placeholders, **five** values. The last one is `id` from the URL. An `UPDATE` does **not** give you `insertId` — there is nothing newly inserted. The JSON the class sent back is just `{ message: "product updated" }`.

```
PUT /product/4
  body: { name: "HP Mouse", price: 1200, description: "...", qty: 20 }

  UPDATE product
     SET name=?, price=?, description=?, qty=?
   WHERE id=?
  values: ["HP Mouse", 1200, "...", 20, "4"]
```

> **Pitfall:** Reading `const { id } = request.params` is not enough. If you forget to put `id` in the values array, MySQL sees four values for five `?` marks. Class hit exactly that 500 — details in Axios PUT and the Missing-ID Bug.

Register a `PUT` on the product router

```
productRouter.put("/product/:id", update);
```

`put` means the backend now accepts the HTTP method the frontend will send. `:id` is the URL parameter Express will parse into `request.params.id`. The handler is the `update` export from `ProductController.js`.

> **Exam:** Update = `PUT /product/:id`. Body = new columns. URL = which row. SQL = `UPDATE … SET ? … WHERE id=?` with **all** placeholders bound, including `id`. Response has no `insertId`.

Edit Flow and useNavigate with ID

Problem: the green pencil must open a form that already knows the product

The list already has an **Edit** `IconButton` (green `EditIcon`, morning leftover). Tonight that click must:

1. Send the user to a **new screen**.
2. Show the **same four fields** as create, already filled with that row.
3. Let the user change any field.
4. On **Update Product**, call the new PUT API.

Create vs update is the same form shape with one difference:

	Create	Update
Fields on first paint	Blank	Prefill from the server
Button label	Add Product	Update Product
HTTP call	`POST /product`	`PUT /product/:id`
After success	Clear the form	**Keep** the current values

The lecturer: *at the time of creation your input fields are blank; at the time of update they are going to be prefilled with the existing product data.* Click Edit on Mouse and Name should already say `mouse`, Description `wireless mouse`, Price `654`, Quantity `13`. Click Edit on Data Cable and those four boxes should show **that** row instead.

Mechanism: a new page, a new route, then `navigate` with the id

They *could* have reused `CreateProduct`. For the session they made a separate page: `src/pages/UpdateProduct/UpdateProduct.tsx`. Memo / `useMemo` / `useCallback` were **deliberately skipped** — the lecturer said he expects you to add those yourself now.

`AppRouter` maps the new URL to that page. The first sketch was a **static** path:

```
<Route path="/edit-product" element={<UpdateProduct />} />
```

Edit's `onClick` used `useNavigate` (Day 13) to send every row to that same URL:

```
const navigate = useNavigate();

<IconButton onClick={() => navigate("/edit-product")}>
  <EditIcon color="success" />
</IconButton>
```

That opens the screen. It does **not** say *which* product. Mouse, Data Cable, Charger and Headphone all land on `/edit-product`. The new component cannot fetch the right row and cannot build `PUT /product/:id`.

Why the id must ride on the frontend URL

Two jobs need that id **before** the PUT is even written:

1. **Prefill** — `GET /product/:id` so the boxes show the current row.
2. **Save** — `PUT /product/:id` so MySQL updates the right `WHERE`.

Frontend routes can take parameters the same way Express does. Change the registration to:

```
<Route path="/edit-product/:id" element={<UpdateProduct />} />
```

and the click to a template string:

```
navigate(`/edit-product/${product.id}`);
```

Live checks in class:

Row clicked	Browser URL
Mouse	`/edit-product/4`
Data Cable	`/edit-product/5`

Row clicked	Browser URL
25 W charger	`/edit-product/7`
Headphone	`/edit-product/8`

```
[ ProductsList ]
      |  Edit on Mouse (id 4)
      |  navigate(`/edit-product/${product.id}`)
      v
[ /edit-product/4 ]
      |
      |  Route path="/edit-product/:id"
      v
[ UpdateProduct ]  -- can now read 4 from the URL
```

Pitfall: `navigate("/edit-product")` plus a route with no `:id` opens a form that cannot know the row. The id is not a backend-only idea — the **React** URL has to carry it too.

Exam: Edit click = `navigate("/edit-product/" + product.id)`. The matching `Route` is `path="/edit-product/:id"`. Same parameter idea as Express `/product/:id`, but this URL lives in the SPA.

---

useParams and Prefill

Problem: the URL has the id; the component has not read it

`UpdateProduct` is the element that opens on `/edit-product/:id`. It must pull `id` out of the current URL. On the server that object was `request.params`. In React Router DOM the hook is **`useParams`**.

The docs line the lecturer read: *returns an object of key-value pairs of the dynamic parameters from the current URL.*

```
import { useParams } from "react-router-dom";

const { id } = useParams();
```

You may write `id` only because the route named the parameter `id`. A path `/edit-product/:productId` would produce `{ productId: "4" }` and `const { id }` would be `undefined`.

Multiple parameters are legal (`:id` and `:slug` together). Tonight there is one.

First live proof — concatenate `id` into a heading. URL `…/8` showed **8**; Edit on Mouse showed **4**; Edit on the 25 W charger showed **7**. Only then did they paste the create form in.

Mechanism: empty state, fetch on load, write the row into `formData`

The form is a copy of `CreateProduct` : same `IFormData` , same `handleChange` , same Grid of `TextField` s, `Box component="form"` . Imports reused from morning: `IFormData` from `CreateProduct/types` , `apiClient` , the two toast helpers. Button text becomes **Update Product**. Alert text becomes **Edit and Update the product**.

State still **starts blank**. Prefill is a second step: when this page mounts, call the API you already have — `GET /product/:id` — and `setFormData` with the JSON.

That fetch belongs in `useEffect` with an **empty** dependency array so it runs once after the first paint (Day 12: "at the time of component load"). You cannot mark the effect callback `async` and `await` inside it cleanly, so class wrapped the work in `fetchById` and called that.

```
const fetchById = async () => {
  try {
    const response = await apiClient.get(`/product/${id}`);
    const productData: IProduct = response.data;
    setFormData({
      name: productData.name,
      description: productData.description,
      price: productData.price.toString(),
      qty: productData.qty.toString(),
    });
  } catch (error) {
    console.log(error);
    showErrorToast("Unable to fetch the data !");
  }
};

useEffect(() => {
  fetchById();
}, []);
```

`IProduct` (from `ProductsList/types`) is the **server** shape: `price` and `qty` are `number` . `IFormData` stores them as **string** because the inputs are controlled text boxes. TypeScript rejected `number` into those keys (*Number is not assignable to string*). Class first tried string concat, then `toString()` on both fields.

Once `formData` is set, the existing `value={formData.name}` bindings do the rest. No extra "put this into the input" API. The lecturer: *you are fetching the data from the server and assigning that*

data into the form state — that's it.

```
mount UpdateProduct
        |
        |  useParams → id
        |  useEffect([]) → fetchById
        |  GET /product/4
        v
  { name, description, price, qty }   // IProduct, numbers on price/qty
        |
        |  toString() on price and qty
        |  setFormData( ... )
        v
  TextFields already show formData via value={}
```

Live checks: Edit Mouse → four boxes show Mouse. Back, Edit Data Cable → that row. Back, Edit 25 W charger → that row.

`handleChange` is already written. The user can now edit any box; that is the same spread + `[event.target.name]` as morning.

Pitfall: `GET /product` (no id) returns the **whole** table. Prefill needs `GET /product/${id}`. And `IProduct.price` is a number — assign it to `IFormData.price` only after `toString()` (or a matching type).

Exam: `useParams` is frontend `request.params`. Prefill = `useEffect` + empty deps + `GET /product/:id` + `setFormData`. The inputs do not need a second binding; they already read `formData`.

Axios PUT and the Missing-ID Bug

Mechanism: same submit handler, different verb and URL

`handleSubmit` still `preventDefault`s. The only API change is `put` instead of `post`, and the path includes the id from `useParams`:

```
const response = await apiClient.put(`/product/${id}`, formData);
showSuccessToast("Product Updated !");
// do not setFormData back to blanks
```

Call	URL	Body
Create	`POST /product`	`formData`
Update	`PUT /product/${id}`	`formData`

After create, morning **cleared** the form so the next insert starts empty. After update you must **not**. The lecturer: *we should not clear the form data because that current form data should still be visible.* Class left the reset line commented out.

Failure toast: **Product Update Failed !**. Fetch failure (above) is a different string: **Unable to fetch the data !**.

The live 500: four values, five question marks

First submit (Mouse → HP Mouse, price `1200`, qty `20`) returned **500**. Debug order the lecturer insisted on:

1. Frontend console — Axios rejected; the API "broke".
2. Backend terminal — `Something went wrong`, then the SQL error: the `id` value never reached the query.
3. `ProductController.update` — `id` was destructured from `request.params` and **not** appended to the values array.

```
SQL:  UPDATE product SET name=? price=? description=? qty=? WHERE id=?
      1           2      3             4            5

First run:  [name, price, description, qty]
            1     2      3             4
            id missing
```

They added `id` as the fifth argument, restarted the API, retried. Toast: **Product Updated !**. Back on the list: **HP Mouse**, price **1200**, quantity **20**.

> **Exam:** Frontend PUT = `await apiClient.put("/product/" + id, formData)`. Do not reset state on success. If the UI gets 500, read the **server** log next — class's bug was a missing bind, not a React mistake.

> **Pitfall:** `console.log` stays during development. The lecturer: when the code is going to merge, strip those logs before the PR. Until then, log on **both** sides so you can walk frontend → backend → SQL.

Data Grids, AG Grid and What This CRUD Is Not

A second, more common edit UX

A separate edit **page** is not the only way. In many real apps the table itself is editable: double-click the Name cell, an input appears **in that cell**, Enter calls the same update API. Searching, sorting, filtering and **pagination** also sit on that table.

Plain HTML / MUI `Table` will not give you that. Industry code uses a **data grid**. The library named in class is **AG Grid** (speech-to-text also heard "AJ Grid" / "EasyGrid" — same product). It is widely used, talks to any major framework, and is highly customisable (sort, filter, and more). The free tier is **limited**; paid features exist. The lecturer's ask: if you have time, explore how to integrate it — useful for the project and for experience.

We never really show data in a plain HTML table in real-time applications; we used to show our data in a data-grid kind of thing.

This CRUD is a beginner step

Create, read, update, delete are now wired end to end — **your** Express APIs to **your** React screens. That is the point of the week, not a claim that the app is production.

The lecturer: *this is really a beginner step. This is not an industry-grade application. Don't think aham brahmasmi.* Still to come in a real system: more security (DDoS, SQL injection — already started with `?` binds), CSRF tokens, a stronger auth story. Enough for the learning curve; not everything.

Tailwind CSS

Problem: MUI gave you components; making them look *yours* still means CSS

The catalogue UI is MUI: `AppBar`, `Drawer`, `TextField`, `Grid`, `Dialog`, `Table`. You can restyle those with your own CSS (spacing, colour, background, animation, margin, padding, font-size, shadow, alignment). Writing that CSS by hand is slow.

Mechanism: utility-first classes, not a second component library

Tailwind CSS ships **hundreds of predefined class names**. You put those names on the element. Tailwind already wrote the CSS behind `flex`, `pt-4`, `text-center`, `rotate-90`, `bg-white`, `font-bold`. You do not start a stylesheet from scratch.

Official line from the site, read in class: *a utility-first CSS framework packed with classes like* `flex`, `pt-4`, `text-center`, `rotate-90`.

What Tailwind is **not**:

Library	What it hands you
MUI / Bootstrap	Ready **components** — Button, TextField, Navbar, Sidebar, Dialog
Tailwind	Ready **utility classes** — background, padding, radius, shadow, font, flex

No Tailwind `<Button>`. No Tailwind navbar. If you skip MUI and use only Tailwind, you write the HTML for navbar / drawer / dialog yourself. Class does not do that. *Components kisne diye? MUI ne — and we are going to enhance those components further with Tailwind.*

Analogies from the lecture (not replacements, **together**):

- Tailwind is **cherry on the cake** — MUI already built the UI; Tailwind makes it more attractive.
- MUI is a beauty parlour; Tailwind is another parlour sitting **on top**.
- You get a free hand to customise what MUI already rendered.

Do **not** memorise every class. There are hundreds. Use the docs; the ones you use every day will stick.

`class` vs `className`

HTML: `class="bg-white"`. React (MUI `Box`, `Typography`, native `div`, your own component): the prop is `className`. There is no `class` attribute in JSX.

```
<Box className="bg-white p-8">
```

> **Exam:** Tailwind is utility-first CSS, not a component kit. MUI + Tailwind work together. In React the prop is `className`.

Tailwind Install on Vite

The app already exists, so this is **add Tailwind to an existing Vite React project**, not `create-vite` from zero. Four steps, in this order. Class followed the Tailwind v4 + Vite docs (the transcript heard `@tailwindcss/v2`; the package they installed and the code is `@tailwindcss/vite`).

1. Install the two packages

Stop `npm run dev`, then:

```
npm install tailwindcss @tailwindcss/vite
```

2. Register the Vite plugin

`vite.config.ts` already has the React plugin. Add Tailwind next to it. Without this plugin the classes you write will not be compiled into CSS.

```
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

3. Import Tailwind from `index.css`

Think of Tailwind as an **external stylesheet**. A page can use those classes only if that sheet is linked. Wipe the default Vite CSS and leave:

```
@import "tailwindcss";
```

4. `main.tsx` must import that CSS

`main.tsx` already has `import './index.css'`. If it did not, no component would see Tailwind. The chain is:

```
npm packages
    → vite.config.ts  plugins: [react(), tailwindcss()]
    → index.css       @import "tailwindcss"
    → main.tsx        import "./index.css"
    → every component may use className="…"
```

> **Pitfall:** Install without the Vite plugin, or import Tailwind in a CSS file that `main.tsx` never loads, and `className="bg-white"` is a string with no styles behind it.

Spacing scale used in class

Tailwind's default spacing step is **4 px**. The number after the hyphen is the multiplier.

Class	Meaning in class	Pixels
`p-1`	padding all sides, 1 × 4	4

Class	Meaning in class	Pixels
`p-2`	2 × 4	8
`p-8`	8 × 4	32
`p-6`	used on the list header	24
`mt-2`	margin-top 2 × 4	8
`mt-8`	margin-top	32
`mb-4`	margin-bottom	16
`gap-4`	flex/grid gap	16

Same idea for `ml-`, `mr-`, `pt-`, `w-12` (width 3 rem / 48 px on the default scale), `h-12`, `h-full`, `min-h-screen`, `w-64`.

Colour, type, radius, shadow — the classes they actually typed

Job	Classes used tonight
White / page grey / indigo theme	`bg-white`, `bg-gray-100`, `bg-indigo-600`, `bg-indigo-100`
Text colour	`text-gray-800`, `text-gray-600`, `text-gray-500`, `text-white`, `text-indigo-600`
Grey shade	`100` light → `800` near-black; heading `800`, body `500`
Weight / size	`font-bold`, `font-semibold`, `text-3xl`, `text-2xl`, `text-lg`, `text-sm`
Radius	`rounded-xl`, `rounded-lg`, `rounded-full` (circle)
Shadow	`shadow-lg`, `shadow-md`
Flex centre	`flex items-center justify-center`
Fill the viewport	`min-h-screen`
Drawer width + scroll	`w-64`, `h-full`, `overflow-auto`

Docs, not memory, when you forget a name (`rounded-xl`, `h-full`, `min-h-screen` were looked up live).

CSS Layers

Problem: the first Dashboard classes did nothing

After install they put `font-bold text-gray-800 text-3xl` on the welcome `Typography`. The text rendered; **Tailwind did not**. The lecturer suspected a **conflict between MUI's CSS and Tailwind's CSS** — two stylesheets, no agreed winner.

Mechanism: tell the browser the layer order

MUI's fix in this session: wrap the app in `StyledEngineProvider` with `enableCssLayer`, and declare a `@layer` list in `GlobalStyles`. Last name in that list wins when two rules fight.

```
import { GlobalStyles, StyledEngineProvider } from "@mui/material";

<StyledEngineProvider enableCssLayer>
  <GlobalStyles styles="@layer theme, base, mui, components, utilities;" />
  <BrowserRouter>
    <AppRouter />
    <ToastContainer />
  </BrowserRouter>
</StyledEngineProvider>
```

```
theme  <  base  <  mui  <  components  <  utilities
                                            ^
                                     Tailwind (last = wins)
```

The lecturer: *if you don't write this, you can't say who is the sole winner — MUI or Tailwind.* After this wrapper, `font-bold` / `text-3xl` showed on the heading.

Layer (in order)	Role in the sentence they used
`theme`	First; later layers may override it
`base`	Overrides theme
`mui`	MUI's own styles override base
`components`	Component styles override MUI defaults
`utilities`	Tailwind utilities override everyone above

> **Exam:** MUI + Tailwind in one Vite app needs `StyledEngineProvider enableCssLayer` and `@layer theme, base, mui, components, utilities`. **Utilities last** so a `className` can override MUI.

> **Pitfall:** Packages installed, plugin registered, `index.css` imported — and still no styles. Check this provider **before** rewriting class names. Class's first miss was the layer order, not a typo in `text-3xl`.

Dashboard, AppLayout and Theme

Page chrome: grey canvas, full viewport

`AppLayout` is the shell around **every** child route (`Dashboard`, `ProductsList`, `CreateProduct`, `UpdateProduct`). Class dropped the old `sx` background and used Tailwind on that outer `Box`:

```
<Box className="bg-gray-100 p-8 min-h-screen">
  <Toolbar />
  <Outlet />
</Box>
```

Class	Why
`bg-gray-100`	Light grey page; cards can then be white
`p-8`	Uniform padding
`min-h-screen`	At least the **visible** viewport tall. Without it the grey only wrapped the content height and stopped short.

Welcome card

A white `Box` (`bg-white p-8 rounded-xl shadow-lg`) holds two `Typography`s: **Welcome to CRUD UI** (`font-bold text-gray-800 text-3xl`) and a lorem line (`mt-2 text-gray-500`). MUI still supplies `Box` / `Typography`; Tailwind only restyles them.

Three equal cards (`4 + 4 + 4`)

Morning Grid again: one `Grid container` (`spacing={4}`, `className="mt-8"`), three children `size={4}`. Each child is a white card:

```
[#### Add (4) ####][#### Add (4) ####][#### Add (4) ####]
```

Inside each card, in order: circular icon, heading, one-line help, contained button.

Circular icon box — this is the pattern they reused on the product list:

```
<Box className="w-12 h-12 rounded-full bg-indigo-100 text-indigo-600 flex items-center
justify-center">
  <AddCircleIcon />
</Box>
```

Class	Job
`w-12 h-12`	Square
`rounded-full`	Square → circle
`bg-indigo-100`	Light indigo disc
`text-indigo-600`	Darker icon (an icon is styled as a font)
`flex items-center justify-center`	Centre on both axes

They first set `items-center justify-center` **without** `flex`. The icon sat on the disc but not in the middle. `flex` is what makes those two alignment classes work.

The three cards were **copied** with the same "Add New Product" copy. The lecturer said change icon / heading / paragraph / button per card as you need, and **you** own the `onClick` (navigate to `/add-product`, `/products-list`, …). That wiring was left as an exercise.

One indigo theme

Same shade **600** on:

- Dashboard buttons — `className="mt-4 rounded-lg bg-indigo-600"`
- `ApplicationTopBar` `AppBar` — `className="bg-indigo-600"`
- Icon discs — `bg-indigo-100` / `text-indigo-600`
- Later: selected sidenav row and the list-header left border

> **Exam:** `min-h-screen` fills the viewport; content-height alone will not. A circle is `w-* h-*` (equal) + `rounded-full` + `flex items-center justify-center`. Dashboard cards are Grid `size={4}` three times.

Active Sidenav

Problem: the drawer does not say which page you are on

If the URL is `/`, **Dashboard** should look selected. On `/add-product`, **Create Product**. On `/products-list`, **Products List**. Otherwise the user has no "you are here".

First attempt: MUI `selected`

`ListItemButton` has a boolean `selected`. True when the browser path equals that item's `path`.

The current path is **not** a string from `window` that you parse by hand. React Router DOM gives `useLocation`, which returns a **location object**. The path is `location.pathname`.

```
const location = useLocation();

<ListItemButton
  selected={location.pathname === menuItem.path}
  onClick={() => {
    navigate(menuItem.path);
    onDrawerClose();
  }}
>
```

`drawerMenu` already stores `{ text, path, icon }` (`/`, `/add-product`, `/products-list`). When `location.pathname === menuItem.path`, that row is the active one. Live: Dashboard highlights on `/`; Create Product on `/add-product`; Products List on `/products-list`. The default MUI highlight looked ugly — good enough to prove the condition.

Then: conditional `className` instead of `selected`

They wanted the **indigo** chip, not MUI's selected grey. `selected` was commented out. The class list becomes a ternary — hardcoded classes cannot change with the URL.

```
<ListItemButton
  className={location.pathname === menuItem.path
    ? "rounded-lg bg-indigo-600 text-white"
    : "rounded-lg text-gray-600"}
>
  <ListItemIcon
    className={location.pathname === menuItem.path
      ? "text-white"
      : "text-indigo-600"}
  >
    {menuItem.icon}
  </ListItemIcon>
```

```
  <ListItemText>{menuItem.text}</ListItemText>
</ListItemButton>
```

State	Row	Icon
Active (`pathname === path`)	`rounded-lg bg-indigo-600 text-white`	`text-white`
Idle	`rounded-lg text-gray-600`	`text-indigo-600`

Idle: grey label, indigo icon, white drawer. Active: indigo background, **white** label **and** white icon.

Drawer chrome

```
<Box className="w-64 pt-15 h-full bg-white overflow-auto">
```

Class	Job
`w-64`	Fixed drawer width
`pt-15`	Space under the fixed AppBar
`h-full`	Full height
`bg-white`	White panel on the grey page
`overflow-auto`	Scroll **inside** if the menu grows; do not stretch the drawer

Exam: `useLocation()` → `location.pathname`. Compare to `menuItem.path`. Either MUI `selected={...}` or a **conditional** `className`. Active indigo row needs the icon recolored too.

Pitfall: Styling the text white and leaving the icon on `text-indigo-600` makes the active row look half-themed. Two ternaries (button + icon), same condition.

Products-List Header

Problem: the blue `Alert` above the table looked cheap next to the new dashboard

They replaced that `Alert` with a header card: icon + title + one-line description, same family as the dashboard welcome / cards.

```
<Box className="p-6 bg-white rounded-xl shadow-md border-l-4 border-indigo-600 flex items-center gap-4">
  <Box className="w-12 h-12 rounded-full bg-indigo-100 text-indigo-600 flex items-center justify-center">
    <ListAltIcon />
  </Box>
  <Box>
    <Typography className="text-2xl font-bold text-gray-800">
      List of Products
    </Typography>
    <Typography className="text-sm text-gray-500">
      View your all of the products here
    </Typography>
  </Box>
</Box>
```

Class	Job
`p-6`	Inner space (first paint had **no** padding and looked crushed; this was the fix)
`bg-white rounded-xl shadow-md`	Card on the grey canvas
`border-l-4 border-indigo-600`	Thick **left** indigo bar
`flex items-center gap-4`	Icon and text in one row, vertically centred, 16 px between them

`border-l-4` is left only. `border-t-*` / `border-b-*` / `border-r-*` are the other sides. Thickness is the number (`1` – `4` …). Colour is a second class (`border-indigo-600`).

Abhay's question: *this colour on the side — how did it arrive?* It is not a second box. It is `border-l-4 border-indigo-600` . `rounded-xl` also rounds that left edge, so the stripe follows the card's corner.

Edit is unchanged except the navigate already discussed: `navigate("/edit-product/" + product.id)` .

> **Exam:** A left accent = `border-l-4` + `border-indigo-600`. `flex items-center gap-4` puts icon and titles on one row. Padding on the **outer** card (`p-6`) is what stops the header looking glued to its own border.

Combined Worked Example

Evening stack after both halves: Express `PUT /product/:id`, edit URL `/edit-product/:id`, prefilled `UpdateProduct`, Tailwind on Vite, CSS layers, indigo theme, active drawer, list header.

`crud-apis/src/controllers/ProductController.js` — `update` only

```
export async function update(request, response) {
  try {
    const { name, price, description, qty } = request.body;
    const { id } = request.params;

    const updateQry =
      `UPDATE product SET name=?, price=?, description=?, qty=? WHERE id=?`;

    const [result] = await sqlConnectionPool.query(updateQry, [
      name, price, description, qty, id,
    ]);

    response.send({ message: "product updated" });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "Something went wrong" });
  }
}
```

`crud-apis/src/routers/ProductRouter.js`

```
import { Router } from "express";
import { save, getAll, getById, deleteById, update } from
"../controllers/ProductController.js";

const productRouter = Router();

productRouter.post("/product", save);
productRouter.get("/product", getAll);
productRouter.get("/product/:id", getById);
productRouter.delete("/product/:id", deleteById);
productRouter.put("/product/:id", update);
```

```
export default productRouter;
```

`crud-ui/src/components/AppRouter/AppRouter.tsx`

```
<Routes>
  <Route path="/" element={<AppLayout />}>
    <Route path="/" element={<Dashboard />} />
    <Route path="/products-list" element={<ProductsList />} />
    <Route path="/add-product" element={<CreateProduct />} />
    <Route path="/edit-product/:id" element={<UpdateProduct />} />
  </Route>
</Routes>
```

Edit cell inside `ProductsList`

```
const navigate = useNavigate();

<IconButton
  onClick={() => {
    navigate(`/edit-product/${product.id}`);
  }}
>
  <EditIcon color="success" />
</IconButton>
```

`src/pages/UpdateProduct/UpdateProduct.tsx`

```
import { Alert, Box, Button, Grid, TextField } from "@mui/material";
import { useEffect, useState, type ChangeEvent, type SubmitEvent } from "react";
import { useParams } from "react-router-dom";
import type { IFormData } from "../CreateProduct/types";
import { apiClient } from "../../api-client/ApiClient";
import { showErrorToast, showSuccessToast } from "../../utils/ToastUtil";
import type { IProduct } from "../ProductsList/types";

export function UpdateProduct() {
  const { id } = useParams();
  const [formData, setFormData] = useState<IFormData>({
    name: "", description: "", price: "", qty: "",
  });

  const fetchById = async () => {
    try {
      const response = await apiClient.get(`/product/${id}`);
      const productData: IProduct = response.data;
      setFormData({
```

```
        name: productData.name,
        description: productData.description,
        price: productData.price.toString(),
        qty: productData.qty.toString(),
      });
    } catch (error) {
      console.log(error);
      showErrorToast("Unable to fetch the data !");
    }
  };

  useEffect(() => {
    fetchById();
  }, []);

  const handleChange = (
    event: ChangeEvent<HTMLInputElement | HTMLTextAreaElement>,
  ) => {
    setFormData({ ...formData, [event.target.name]: event.target.value });
  };

  const handleSubmit = async (event: SubmitEvent<HTMLFormElement>) => {
    event.preventDefault();
    try {
      const response = await apiClient.put(`/product/${id}`, formData);
      console.log(response.data);
      showSuccessToast("Product Updated !");
    } catch (error) {
      console.log(error);
      showErrorToast("Product Update Failed !");
    }
  };

  return (
    <Box>
      <Alert severity="success">Edit and Update the product</Alert>
      <Box component="form" onSubmit={handleSubmit} sx={{ marginTop: "30px" }}>
        <Grid container rowSpacing={4} columnSpacing={5} sx={{ mb: 4 }}>
          <Grid size={4}>
            <TextField label="Name" name="name" value={formData.name}
              onChange={handleChange} fullWidth />
          </Grid>
          <Grid size={4}>
            <TextField label="Description" name="description"
              value={formData.description} onChange={handleChange} fullWidth />
          </Grid>
        </Grid>
        <Grid container rowSpacing={4} columnSpacing={5} sx={{ mb: 4 }}>
          <Grid size={4}>
            <TextField label="Price" name="price" value={formData.price}
              onChange={handleChange} fullWidth />
```

```
          </Grid>
          <Grid size={4}>
            <TextField label="Quantity" name="qty" value={formData.qty}
              onChange={handleChange} fullWidth />
          </Grid>
        </Grid>
        <Button type="submit" variant="contained">Update Product</Button>
      </Box>
    </Box>
  );
}
```

Tailwind wiring

`package.json` dependencies include `tailwindcss` and `@tailwindcss/vite`. `vite.config.ts` plugins are `[react(), tailwindcss()]`. `src/index.css` is only `@import "tailwindcss";`. `src/main.tsx` imports `./index.css`.

`src/App.tsx` — layers so Tailwind can win

```
import { BrowserRouter } from "react-router-dom";
import { AppRouter } from "./components/AppRouter/AppRouter";
import { ToastContainer } from "react-toastify";
import { GlobalStyles, StyledEngineProvider } from "@mui/material";

export default function App() {
  return (
    <StyledEngineProvider enableCssLayer>
      <GlobalStyles styles="@layer theme, base, mui, components, utilities;" />
      <BrowserRouter>
        <AppRouter />
        <ToastContainer />
      </BrowserRouter>
    </StyledEngineProvider>
  );
}
```

End-to-end trace — update Mouse

1. List is on `/products-list`. `useProductsList` already loaded `GET /product`. Mouse is id **4**.
2. Green pencil runs `navigate("/edit-product/" + product.id)`. Browser URL becomes `/edit-product/4`.
3. `AppRouter` matches `path="/edit-product/:id"` and mounts `UpdateProduct` inside `AppLayout` (`Outlet`). Grey `min-h-screen` canvas is already around the page.

4. `useParams()` is `{ id: "4" }`. `formData` starts as four empty strings, so the first paint is blank boxes.
5. `useEffect([])` calls `fetchById`. `apiClient.get("/product/4")` hits Express `getById`. `response.data` is the Mouse JSON. `price` / `qty` go through `toString()`. `setFormData` fills the four `TextField`s.
6. User changes name to **HP Mouse**, price to **1200**, qty to **20**. `handleChange` spreads + `[event.target.name]`.
7. **Update Product** submits the `Box component="form"`. `preventDefault`, then `apiClient.put("/product/4", formData)`.
8. Express `update` reads the body and `request.params.id`, runs `UPDATE product SET … WHERE id=?` with **five** binds. Replies `{ message: "product updated" }`.
9. `showSuccessToast("Product Updated !")`. Fields stay on HP Mouse / 1200 / 20 (no reset).
10. Back to `/products-list`. A refetch (or a revisit that mounts the list again) shows HP Mouse, 1200, 20. The drawer row **Products List** is indigo because `useLocation().pathname === "/products-list"`.

End-to-end trace — the 500 they actually hit

Step 7 the first time: frontend saw 500. Server log: SQL missing the `id` bind. They appended `id` to the values array, restarted `index.js`, repeated steps 6–9. Same UI, successful UPDATE.

End-to-end trace — first Tailwind miss

Welcome `Typography` had `font-bold text-3xl` and still looked like default MUI. Packages and the Vite plugin were already in. Wrapping `App` in `StyledEngineProvider enableCssLayer` + the `@layer … utilities` line made those classes win. Then `AppLayout` `bg-gray-100`, the white welcome card, three `size={4}` cards, indigo `AppBar` / buttons, conditional sidenav, list header with `border-l-4`.

Quick Revision Checklist

Topic	One-line takeaway
Update needs two inputs	Body = new columns; URL `:id` = which row
SQL	`UPDATE product SET name=?, price=?, description=?, qty=? WHERE id=?`
Binds	**Five** values, last is `id`. Forget it → 500

Topic	One-line takeaway
`insertId`	Does not exist on UPDATE; send `{ message: "product updated" }`
Router	`productRouter.put("/product/:id", update)`
Edit UX	New page, prefilled form, **Update Product**, then PUT
Create vs update	Blank + POST + clear vs prefill + PUT + **keep** values
Route	`/edit-product/:id` → `UpdateProduct`
`navigate`	`navigate("/edit-product/" + product.id)` — id **must** be in the SPA URL
`useParams`	Frontend `request.params`; key = the name after `:`
Prefill	`useEffect` + `[]` + `GET /product/${id}` + `setFormData`
`toString()`	`IProduct` numbers → `IFormData` strings
PUT call	`apiClient.put("/product/" + id, formData)`
Debug 500	Console first, **then** server log, then the query binds
AG Grid	Real apps use a data grid (sort / filter / inline edit); this table is a beginner step
Tailwind	Utility classes, **not** MUI-style components
Together	MUI builds the widget; Tailwind restyles it (`className`)
Install	`tailwindcss` + `@tailwindcss/vite` → plugin → `@import "tailwindcss"` → `main.tsx`
Spacing	Number × **4 px** (`p-8` = 32 px)
Layers	`StyledEngineProvider enableCssLayer` + `@layer theme, base, mui, components, utilities`
Last layer	`utilities` (Tailwind) overrides MUI
`min-h-screen`	Grey canvas fills the viewport, not just the content height
Circle icon	`w-12 h-12 rounded-full flex items-center justify-center` — `flex` is required
Theme	`bg-indigo-600` on AppBar, buttons, active nav, left border
`useLocation`	`location.pathname === menuItem.path`

Topic	One-line takeaway
Active nav	Conditional `className` (indigo + white text/icon) or MUI `selected`
List header	`border-l-4 border-indigo-600` is Abhay's left stripe
Homework tone	Memo on `UpdateProduct`; card button `onClick`s; explore AG Grid + Tailwind docs

---

Practice Exercises

Concept

1. An update query needs two pieces of data. What are they, and where does Express read each one?
2. Why must the `UPDATE` use `?` placeholders instead of `` `SET name='${name}'` ``?
3. How many placeholders are in the class SQL, and which value is easy to forget in the binds array?
4. Why did the lecturer refuse to send `insertId` in the update response?
5. Write the Express registration line for the new API (method, path, handler name).
6. How does the update **screen** differ from create on first paint, on the button label, and after a successful save?
7. Why is `navigate("/edit-product")` not enough, even if `UpdateProduct` mounts?
8. Give the two reasons the product id must be on the **frontend** URL.
9. What does `useParams` return, and why could class write `const { id } = useParams()`?
10. `useParams` is the frontend cousin of which Express object?
11. Why is the GET that prefills the form `/product/${id}` and not `/product`?
12. Why did `setFormData({ … price: productData.price })` fail TypeScript, and what did class call to fix it?
13. Why wrap the fetch in `fetchById` instead of putting `await` on the `useEffect` callback?
14. After a successful PUT, why is `setFormData({ name: "", … })` the wrong copy-paste from create?
15. First update returned 500. In what **order** did class debug, and what was the actual fault?
16. What extra UX does the lecturer describe instead of a separate edit page, and which library did he name for sort / filter / pagination?

17. Tailwind vs MUI: who gives components, who gives utility classes, and do they replace each other?
18. In React, why is `class="bg-white"` the wrong way to apply a Tailwind class?
19. List the four install/wiring steps for Tailwind on this existing Vite app.
20. What does the `4` in `p-4` mean in pixels on the default scale?
21. Why did `font-bold text-3xl` first do nothing, and which two MUI pieces fixed the winner?
22. In `@layer theme, base, mui, components, utilities`, which layer wins a clash?
23. Why `min-h-screen` on `AppLayout` and not only `bg-gray-100`?
24. They set `items-center justify-center` on the icon disc and the icon still sat off-centre. Which class was missing?
25. How do you know the sidenav row should look active, and which hook gives you the current path?
26. Abhay asked how the indigo strip arrived on the list header. Which two classes are that strip?

Application

27. Write `update` (destructure body + params, parameterized SQL, five binds, `{ message: "product updated" }`, 500 on error).
28. Write the `Route` for the edit page and the Edit-icon `onClick` that fills `:id`.
29. Write `fetchById` + the `useEffect` that runs it on mount (GET, `IProduct`, `toString`, error toast).
30. Write the `try/catch` body of update `handleSubmit` after `preventDefault` (PUT, success toast, no reset).
31. Write the `StyledEngineProvider` / `GlobalStyles` wrapper from `App.tsx`.
32. Write the circular indigo icon `Box` around `<AddCircleIcon />`.
33. Write the `ListItemButton` `className` ternary and the matching `ListItemIcon` ternary.

Predict the output

34. Route is `/edit-product/:id`. Click Edit on the row whose `id` is `7`. What is the browser path?
35. Route is `/edit-product/:productId`. Component does `const { id } = useParams()`. What is `id`?
36. `fetchById` calls `apiClient.get("/product")` (no id). What lands in `productData` if you assign `response.data` as `IProduct`?
37. Update SQL has five `?`. Values array is `[name, price, description, qty]`. What does the client see, and where is the real error message?
38. PUT succeeds and the handler also runs the create-page reset. What do the four boxes show?

39. `className="items-center justify-center w-12 h-12 rounded-full bg-indigo-100"` — no `flex`. Where does the icon sit?
40. `App` has no `StyledEngineProvider`. `Typography` has `className="text-3xl font-bold"`. What do you see?
41. `ListItemButton` is always `className="rounded-lg bg-indigo-600 text-white"` (no ternary). What does the drawer look like on every page?

Spot the bug

42. `productRouter.post("/product/:id", update)` — the handler is the new update function. What is wrong for the frontend `apiClient.put`?
43. `navigate("/edit-product/" + product)` where `product` is the whole object. What does the URL look like?
44. `useEffect(async () => { const response = await apiClient.get( ... ) }, [])` as written in a beginner file. What rule from class did they skip?
45. Prefill assigns `price: productData.price` with `IFormData.price: string` and `IProduct.price: number`. What does `tsc` say?
46. `vite.config.ts` still has only `plugins: [react()]`. `index.css` has `@import "tailwindcss"`. Why is the page unstyled?
47. Sidenav colours the **text** with the ternary but `ListItemIcon` is a fixed `className="text-indigo-600"`. What looks wrong on the active row?
48. Header card has `border-l-4` but no `border-indigo-600` (and no other border colour). What happens to Abhay's stripe?

Practice Answers

1. New column values from `request.body` (`name`, `price`, `description`, `qty`). Which row from `request.params.id` (the `:id` on `PUT /product/:id`).

2. Concatenating user input into SQL is the injection hole from Day 8. `?` plus a values array lets the driver bind safely.

3. Five: four `SET` columns and `WHERE id=?`. The forgotten one in class was `id`.

4. Nothing was inserted. `insertId` is an INSERT field; UPDATE does not produce one.

5. `productRouter.put("/product/:id", update);`

6. First paint: create blank, update prefilled. Button: **Add Product** vs **Update Product**. After success: create clears; update **keeps** the current values.

7. Every row would open the same URL. The page cannot fetch or PUT a specific product.

8. (1) `GET /product/:id` to prefill. (2) `PUT /product/:id` to save. Both need the id before/while you are on that screen.

9. An object of the current URL's dynamic params. The route declared `:id`, so the key is `id`.

10. `request.params`.

11. `/product` is **all** rows. Prefill needs **one** JSON object for that id.

12. `price` / `qty` are `number` on `IProduct` and `string` on `IFormData`. Class used `.toString()` on both.

13. The effect callback is not the place they `await`ed. An inner `async` function (`fetchById`) holds the try/catch; the effect only **calls** it.

14. The user should still see the **current** (just saved) values. Clearing is a create habit.

15. Frontend console (API failed) → backend log (`Something went wrong` / SQL) → controller values array. Fault: `id` read from params but not bound.

16. Double-click a cell, edit in place, Enter calls update. **AG Grid** for grid features (sort, filter, pagination, …).

17. MUI (and Bootstrap) give components. Tailwind gives utility classes. They work **together**; Tailwind is not a replacement.

18. JSX has no `class`. The prop is `className`.

19. `npm install tailwindcss @tailwindcss/vite`; add `tailwindcss()` in `vite.config.ts`; `@import "tailwindcss"` in `index.css`; import that CSS from `main.tsx`.

20. `4 × 4 px = 16 px` padding on all sides.

21. MUI and Tailwind were fighting; no agreed winner. `StyledEngineProvider enableCssLayer` plus `GlobalStyles` `@layer theme, base, mui, components, utilities`.

22. `utilities` — last in the list.

23. Without `min-h-screen` the grey only wrapped the content block. They wanted the **whole visible page** grey.

24. `flex`. `items-center` / `justify-center` apply to a flex (or grid) container.

25. When `location.pathname` equals that `menuItem.path`. Hook: `useLocation`.

26. `border-l-4` and `border-indigo-600`. (`rounded-xl` also rounds that left edge.)

27.

```
export async function update(request, response) {
  try {
    const { name, price, description, qty } = request.body;
    const { id } = request.params;
    const updateQry =
      "UPDATE product SET name=?, price=?, description=?, qty=? WHERE id=?";
    await sqlConnectionPool.query(updateQry, [name, price, description, qty, id]);
    response.send({ message: "product updated" });
  } catch (error) {
    console.log(error);
    response.status(500).send({ message: "Something went wrong" });
  }
}
```

28.

```
<Route path="/edit-product/:id" element={<UpdateProduct />} />

onClick={() => navigate(`/edit-product/${product.id}`)}
```

29.

```
const fetchById = async () => {
  try {
    const response = await apiClient.get(`/product/${id}`);
    const productData: IProduct = response.data;
    setFormData({
      name: productData.name,
      description: productData.description,
      price: productData.price.toString(),
      qty: productData.qty.toString(),
    });
  } catch (error) {
    console.log(error);
    showErrorToast("Unable to fetch the data !");
  }
};

useEffect(() => {
  fetchById();
}, []);
```

30.

```
try {
  const response = await apiClient.put(`/product/${id}`, formData);
  console.log(response.data);
```

```
    showSuccessToast("Product Updated !");
  } catch (error) {
    console.log(error);
    showErrorToast("Product Update Failed !");
  }
```

31.

```
<StyledEngineProvider enableCssLayer>
  <GlobalStyles styles="@layer theme, base, mui, components, utilities;" />
  <BrowserRouter>
    <AppRouter />
    <ToastContainer />
  </BrowserRouter>
</StyledEngineProvider>
```

32.

```
<Box className="w-12 h-12 rounded-full bg-indigo-100 text-indigo-600 flex items-center
justify-center">
  <AddCircleIcon />
</Box>
```

33.

```
<ListItemButton
  className={
    location.pathname === menuItem.path
      ? "rounded-lg bg-indigo-600 text-white"
      : "rounded-lg text-gray-600"
  }
>
  <ListItemIcon
    className={
      location.pathname === menuItem.path
        ? "text-white"
        : "text-indigo-600"
    }
  >
    {menuItem.icon}
  </ListItemIcon>
</ListItemButton>
```

34. `/edit-product/7` .

35. `undefined` . The object key is `productId` , not `id` .

36. An **array** of all products (or that array treated as one object). Not a single `{ name, price, … }` row. Prefill will be wrong or crash when you read `.name`.

37. Axios 500 / error toast. The real SQL bind error is on the **server** log. `id` never reached `WHERE`.

38. Four empty strings — the just-saved HP Mouse vanishes from the boxes even though MySQL has it.

39. On the disc but **not centred**. Alignment classes do not apply without `flex` (or grid).

40. MUI's default heading look. Tailwind utilities lose the fight (or never apply). Class saw this before the layer wrapper.

41. Every row looks selected (indigo + white). There is no idle state.

42. Method mismatch. The browser sends **PUT**; Express only registered **POST** on that path. The update handler never runs (or the wrong verb 404s).

43. Something like `/edit-product/[object Object]`. You must send `product.id`, not the object.

44. They `await`ed on the effect itself. Class put `async` on an inner `fetchById` and only **called** it from `useEffect`.

45. `Type 'number' is not assignable to type 'string'` (the live error). Use `.toString()`.

46. The Vite Tailwind plugin never runs. `@import` alone does not compile utilities into the bundle they served.

47. Active row: white label on indigo, but the icon stays **indigo-on-indigo** (low contrast). Icon needs the same condition → `text-white`.

48. You get a 4 px left border in the **default** border colour (or nothing visible), not the indigo accent. Thickness and colour are two classes.